



UNIVERSIDADE
LUSÓFONA

Uma perspectiva de engenharia sobre a evolução de algoritmos de ranqueamento baseada em grafos.

Tales Santos

Dissertação para obtenção do Grau de Mestre em
Mestrado em Engenharia Informática e Sistemas de Informação

Orientador: Aleksandar Mikovic
Professor xxx, Universidade Lusófona

Coorientador: Sofia Fernandes
Professor xxx, Universidade Lusófona

Lisboa, setembro, 2026

Título da tese

Copyright © Tales Eduardo dos Santos, Departamento de XXX, Universidade Lusófona.

O Departamento de XXX e a Universidade Lusófona têm o direito, perpétuo e sem limites geográficos, de arquivar e publicar esta dissertação através de exemplares impressos reproduzidos em papel ou de forma digital, ou por qualquer outro meio conhecido ou que venha a ser inventado, e de a divulgar através de repositórios científicos e de admitir a sua cópia e distribuição com objetivos educacionais ou de investigação, não comerciais, desde que seja dado crédito ao autor e editor.

A epígrafe, a existir, deve conter pensamento(s) ou frase(s) pertinente(s) servindo como abertura do trabalho e/ou das partes/capítulos.

A dedicatória é um elemento opcional, no qual o candidato presta uma homenagem ou dedica o trabalho a alguém.

Agradecimentos

Os agradecimentos, são um elemento opcional, no qual o candidato deverá registrar o reconhecimento às pessoas e/ou instituições que contribuíram de forma relevante para a elaboração do trabalho

Resumo

A priorização de requisitos de software é uma das atividades mais sensíveis da Engenharia de Requisitos: técnicas tradicionais como a Votação Acumulativa ou o PERT/CPM dependem fortemente da percepção subjetiva dos stakeholders e não escalam para sistemas com elevado número de requisitos interdependentes. Esta dissertação investiga uma alternativa de natureza estrutural, assente na aplicação de algoritmos de ranqueamento de grafos — PageRank, HITS e SALSA — ao grafo de dependências entre requisitos.

Foi desenvolvido o **ReqGraph**, um protótipo Python que automatiza o pipeline *Código-fonte* → *AST* → *Call Graph* → *Grafo de Requisitos*, e o módulo **ranker.py**, que aplica os três algoritmos ao grafo extraído. A validação foi conduzida com três projetos reais de código aberto — CPython stdlib, Flask e scikit-learn — cobrindo grafos de 9 a 17 arestas. Os resultados mostram que os algoritmos convergem em casos suficientemente densos (identificando, por exemplo, REQ_PIPELINE como o requisito mais central do scikit-learn) e divergem informativamente em casos esparsos, oferecendo perspectivas complementares de importância (transitiva, direta e de orquestração). Conclui-se que o ranqueamento estrutural constitui um complemento robusto às técnicas tradicionais, contribuindo para a redução do viés subjetivo na priorização de requisitos.

Palavras-chave

Engenharia de Requisitos. Priorização Estrutural. Análise Estática de Código. Grafos. PageRank. HITS. SALSA..

Abstract

Software requirements prioritization is one of the most sensitive activities in Requirements Engineering: traditional techniques such as Cumulative Voting or PERT/CPM rely heavily on the subjective perception of stakeholders and do not scale well to systems with a large number of interdependent requirements. This thesis investigates a structural alternative based on the application of graph ranking algorithms — PageRank, HITS, and SALSA — to the dependency graph between requirements.

A Python prototype, ReqGraph, was developed to automate the pipeline source code $\rightarrow$ AST $\rightarrow$ call graph $\rightarrow$ requirement graph, together with the `ranker.py` module which applies the three algorithms to the extracted graph. Validation was conducted with three real open-source projects — CPython stdlib, Flask, and scikit-learn — covering graphs from 9 to 17 edges. The results show that the algorithms converge in sufficiently dense cases (identifying, for example, REQ_PIPELINE as the most central requirement of scikit-learn) and diverge informatively in sparse cases, offering complementary perspectives on importance (transitive, direct, and orchestration). We conclude that structural ranking constitutes a robust complement to traditional techniques, helping reduce subjective bias in requirements prioritization.

.

Keywords

Requirements Engineering. Structural Prioritization. Static Code Analysis. Graphs. PageRank. HITS. SALSA..

Tabela de Conteúdos

Agradecimentos	vi
Resumo	viii
Abstract	ix
Tabela de Conteúdos	x
Índice de Figuras	xiv
Índice de Tabelas	xvi
Abreviaturas	xviii
Introdução	1
1 Introdução	1
1.1 Enquadramento e Motivação	2
1.2 Problema e Objetivos	2
1.3 Objetivos	3
1.3.1 <i>Objetivos Gerais</i>	3
1.3.2 <i>Objetivos Específicos</i>	3
1.4 Abordagem Metodologica.....	4
1.4.1 <i>Levantamento e Sistematização Teórica</i>	4
1.4.2 <i>Ambiente de Experimentação e Ferramentas</i>	4
1.4.3 <i>Emulação e Modelagem do Grafo de Requisitos</i>	4
1.4.4 <i>Protocolo de Avaliação e Análise</i>	5
2 Referencial Teórico	7
2.1 Conceitos básicos de Grafos	8
2.1.1 <i>Tipos de Grafos</i>	8
2.1.2 <i>Caminhos e passeios aleatórios</i>	12
2.2 Algoritmos de ranqueamento em grafos	12

2.2.1	<i>Centralidade de grau</i>	12
2.2.2	<i>PageRank</i>	12
2.2.3	<i>HITS (Hyperlink-Induced Topic Search)</i>	13
2.2.4	<i>SALSA (Stochastic Approach for Link-Structure Analysis)</i>	14
2.2.5	<i>BiRank e normalizações simétricas</i>	15
2.3	<i>Aplicação das Métricas ao Ranking e Priorização de Requisitos</i>	15
2.3.1	<i>Fundamentos da priorização de requisitos e desafios</i>	15
2.3.2	<i>Construção do grafo de requisitos</i>	16
2.3.3	<i>Mapeamento dos algoritmos de ranqueamento para priorização</i>	16
2.3.4	<i>Benefícios, limitações e considerações práticas</i>	17
3	Metodologia	18
3.1	<i>Estratégia de Investigação</i>	19
3.2	<i>Arquitetura do Protótipo ReqGraph</i>	19
3.3	<i>Implementação e Tecnologias</i>	21
3.3.1	<i>Módulo de Raqueamento (ranker.py)</i>	21
3.3.2	<i>PageRank</i>	21
3.3.3	<i>HITS</i>	21
3.3.4	<i>SALSA</i>	22
3.3.5	<i>Artefatos Gerados</i>	22
3.4	<i>Corpus Experimental e Protocolo de Avaliação</i>	22
3.4.1	<i>Seleção dos Casos de Estudo</i>	22
3.4.2	<i>Protocolo Experimental</i>	23
3.4.3	<i>Dimensões de Análise</i>	24
4	Avaliação de Resultados	25
4.1	<i>Resultados Técnicos</i>	26
4.1.1	<i>Síntese Quantitativa</i>	26
4.1.2	<i>Caso Simples – Cpython stdlib</i>	27
4.1.3	<i>Caso Intermediário – Flask</i>	28
4.1.4	<i>Caso Complexo – scikit-learn</i>	29
4.2	<i>Resultados Analíticos</i>	31
4.2.1	<i>Caso Simples – Rankings para o CPython stdlib</i>	31
4.2.2	<i>Caso Intermediário – Rankings para o Flask</i>	33
4.2.3	<i>Caso Complexo – Rankings para scikit-learn</i>	35
4.3	<i>Comparação Consolidada</i>	36
4.4	<i>Discussão</i>	37
5	Conclusão	38

5.1 Sumário e Principais Contribuições	39
5.2 Trabalho Futuro.....	41
Bibliografia.....	42
Glossário.....	47

Índice de Figuras

*Figura 1 – Indicadores de qualidade e melhoria-continua (1).*_____ ***Erro! Marcador não definido.***

*Figura 2 – Logotipo do DEISI.*_____ ***Erro! Marcador não definido.***

*Figura 3 – Indicadores de qualidade e melhoria-continua (2).*_____ ***Erro! Marcador não definido.***

Índice de Tabelas

<i>Tabela 1 – Tipos de Selectores CSS.....</i>	<i>Erro! Marcador não definido.</i>
<i>Tabela 2 – Tipos de Selectores CSS.....</i>	<i>Erro! Marcador não definido.</i>
<i>Tabela 3 – Tipos de Selectores CSS.....</i>	<i>Erro! Marcador não definido.</i>
<i>Tabela 4 – Tipos de Selectores CSS.....</i>	<i>Erro! Marcador não definido.</i>
<i>Tabela 5 – Tipos de Selectores CSS.....</i>	<i>Erro! Marcador não definido.</i>
<i>Tabela 6 – Tipos de Selectores CSS.....</i>	<i>Erro! Marcador não definido.</i>

Abreviaturas, Siglas e Símbolos

COFAC	Cooperativa de Formação e Animação Cultural
DEISI	Departamento de Engenharia Informática e Ssistemas de Informação
$G = (V, E)$	grafo com vértices V e arestas E
r	Vetor de Ranks
d	Fator de amortecimento
A	Matriz de adjacência
$A_{ij} = 1$	se há aresta de j para i (em grafos direcionados)

Capítulo 1

Introdução

O Capítulo 1 apresenta uma introdução à tese, apresentando, na Secção 1.1, um enquadramento. A motivação é discutida na Secção 1.2, sendo que os objetivos principais são apresentados na secção 1.3. Na As principais questões metodológicas encontram-se descritas na Secção 1.5. Finalmente, os conteúdos do restante documento são apresentados na Secção 1.6.

Conceitos chave: enquadramento; motivação; objetivos; novidade; impacto; visão.

1.1 Enquadramento e Motivação

O desenvolvimento de sistemas de software em larga escala enfrenta desafios crescentes na gestão da complexidade e na organização de grandes volumes de informação interdependente. Tradicionalmente, a relevância de elementos dentro de uma rede era determinada por análises textuais ou heurísticas manuais, mas a evolução da **Teoria dos Grafos** permitiu uma transição para métodos puramente estruturais.

A motivação principal para este estudo surge da convergência entre algoritmos de alta performance utilizados na indústria e a necessidade de rigor analítico na engenharia de software. Dois marcos fundamentais impulsionaram esta investigação:

- **Engenharia Reversa e Algoritmos de Produção:** A análise do código-fonte aberto do sistema de recomendação do **Twitter** (repositório the-algorithm) revelou a aplicação prática de variantes do algoritmo **SALSA** para sugestões de conexão, demonstrando a viabilidade de usar passeios aleatórios em grafos de larga escala para determinar prestígio e relevância.
- **Otimização da Engenharia de Requisitos:** A descoberta de pesquisas recentes que aplicam o **PageRank** para a priorização de requisitos de software. Este trabalho demonstrou que a estrutura de dependências de um sistema contém informações latentes que podem ser usadas para ordenar o desenvolvimento de forma mais eficiente do que métodos subjetivos tradicionais.

Dessa forma, o trabalho enquadra-se na busca por modelos matemáticos que minimizem o viés humano em decisões críticas de engenharia, utilizando propriedades topológicas para extrair inteligência de grafos de software.

1.2 Problema e Objetivos

O ranqueamento de requisitos de software é um problema complexo que exige uma transição de abordagens experimentais e subjetivas para métodos que utilizam a rica informação estrutural do sistema. Os conceitos de Processos Estocásticos, particularmente Passeios Aleatórios e Cadeias de Markov, fornecem a base teórica robusta para algoritmos de ranqueamento como PageRank, HITS e SALSA (Masuda et al., 2017). – “Para o caso de ranqueamento de requisitos”

A aplicação de PageRank, que determina a importância pela distribuição estacionária no grafo de dependências, é demonstrada como um caminho viável e superior à Votação Acumulativa e ao PERT/CPM, especialmente por sua escalabilidade e capacidade de lidar com interdependências (Silva et al., 2018). O estudo comparativo deve, no entanto, considerar as vulnerabilidades topológicas, como

o Efeito TKC, que o SALSA foi especificamente desenvolvido para mitigar (Lempel & Moran, 2001).

Portanto, esta tese se justifica ao investigar e comparar a eficácia do PageRank, HITS e SALSA na priorização estrutural de requisitos (Silva et al., 2018). O objetivo é fornecer um modelo analítico que minimize o esforço dos stakeholders, utilize integralmente a rastreabilidade de requisitos e permita a incorporação controlada de fatores subjetivos através de técnicas como vértices artificiais, garantindo uma priorização mais consistente e robusta, especialmente em grandes sistemas, onde a subjetividade m

1.3 Objetivos

O ranqueamento de requisitos de software é um problema complexo que exige a transição de abordagens puramente experimentais e subjetivas para métodos que utilizam a rica informação estrutural do sistema. A utilização de processos estocásticos, como Passeios Aleatórios oferece uma base teórica robusta para algoritmos de ranking como PageRank, HITS e SALSA.

1.3.1 Objetivos Gerais

Investigar e comparar a eficácia desses algoritmos na priorização estrutural de requisitos, fornecendo um modelo analítico que utilize a rastreabilidade para garantir consistência em grandes sistemas

1.3.2 Objetivos Específicos

- Mapeamento Teórico-Matemático: Formalizar a aplicação de processos estocásticos e propriedades de grafos para modelar interdependências de requisitos e fluxos de influência em redes sociais.
- Avaliação de Eficácia na PRS: Demonstrar, por meio de simulações, como o algoritmo PageRank supera técnicas tradicionais (como Votação Acumulativa e PERT/CPM) ao lidar com a escalabilidade e a resolução de requisitos mutuamente dependentes.
- Análise de Vulnerabilidades Topológicas: Avaliar o comportamento dos algoritmos frente a anomalias estruturais, especificamente o Efeito TKC (Tightly Knit Community), comparando a resiliência do SALSA em relação ao HITS em subgrafos de redes sociais.
- Desenvolvimento de Modelo Híbrido: Propor uma técnica para a incorporação de fatores subjetivos (prioridades de negócio ou técnicas) na estrutura do grafo através da criação de vértices artificiais, permitindo um ajuste flexível e coerente do ranking final.

- **Validação por Emulação:** Implementar um protótipo experimental que emule a estrutura de uma aplicação de larga escala para medir a redução do esforço dos stakeholders e a precisão do ranqueamento automático frente à rastreabilidade de requisitos.

1.4 Abordagem Metodologica

A presente investigação adota uma abordagem quantitativa e experimental, fundamentada na modelagem matemática de sistemas de software através da Teoria dos Grafos. O trabalho será estruturado em quatro etapas procedimentais:

1.4.1 Levantamento e Sistematização Teórica

Realização de uma revisão bibliográfica focada na evolução dos processos estocásticos aplicados ao ranking de links. Esta fase visa consolidar o entendimento sobre as diferenças estruturais entre o **HITS** (dualidade Hubs/Authorities), o **SALSA** (passeios aleatórios em grafos bipartidos) e o **PageRank** (distribuição estacionária em cadeias de Markov).

1.4.2 Ambiente de Experimentação e Ferramentas

Para a execução dos testes, será utilizado o ambiente **Google Colab**, utilizando a linguagem **Python** e a biblioteca **NetworkX**. Esta escolha justifica-se pela necessidade de manipular matrizes de adjacência e realizar cálculos iterativos de autovetores de forma escalável.

1.4.3 Emulação e Modelagem do Grafo de Requisitos

A fase prática envolverá a emulação dos requisitos de uma aplicação de larga escala. Este processo seguirá os seguintes passos:

- **Construção da Topologia:** Mapeamento dos requisitos como vértices (\$V\$) e suas interdependências (rastreabilidade) como arestas direcionadas (\$E\$).
- **Introdução de Variáveis de Negócio:** Implementação de **vértices artificiais** para injetar prioridades técnicas ou financeiras subjetivas, permitindo avaliar a flexibilidade do ranking estrutural.
- **Cenários de Teste:** Geração de subgrafos densos para testar especificamente a sensibilidade ao **Efeito TKC** em cada algoritmo.

1.4.4 Protocolo de Avaliação e Análise

Os resultados serão avaliados comparativamente. A análise focará em três dimensões:

- **Consistência Algorítmica:** Comparação entre os rankings gerados pelos três algoritmos para identificar convergências e discrepâncias.
- **Desempenho Técnico:** Medição do tempo de convergência e esforço computacional em grafos de diferentes magnitudes.
- **Validação Frente a Métodos Tradicionais:** Comparação qualitativa da redução do esforço dos *stakeholders* em relação a métodos manuais, como a Votação Acumulativa.

Capítulo 2

Revisão Bibliográfica – Referencial Teórico

O Capítulo 2 apresentam-se recomendações para escrita de uma tese. Na Secção 2.1, são apresentados conceitos basilares. Na Secção 2.2, são identificados os aspectos críticos. Na Secção 2.3, são apresentados avanços na área em estudo. (Recriar resumo de tópicos)

Conceitos chave: visão geral; conceitos basilares; desafios; avanços; estado da arte.

2.1 Conceitos básicos de Grafos

Um grafo é uma estrutura matemática que modela relações entre objetos. Formalmente, um grafo $G = (V, E)$ é definido por um conjunto de vértices (ou nós) V e um conjunto de arestas E , onde cada aresta é um par não ordenado (ou ordenado) de vértices. Os vértices representam as entidades de interesse, e as arestas representam as relações entre elas. Em redes complexas, os termos "grafo" e "rede" são frequentemente usados como sinônimos.

Exemplo: Considere uma pequena rede de colaboração entre pesquisadores. Os vértices podem ser $\{\text{Ana, Bruno, Carla, Daniel}\}$. As arestas podem representar coautorias: se Ana e Bruno publicaram juntos, existe uma aresta entre eles. Suponha as seguintes colaborações: Ana–Bruno, Ana–Carla, Bruno–Carla, Carla–Daniel. O grafo correspondente tem $V = \{A, B, C, D\}$ e $E = \{AB, AC, BC, CD\}$.

Uma forma comum de representar um grafo é por meio da **matriz de adjacência** A , onde $A_{ij} = 1$ se existe uma aresta entre os vértices i e j , e 0 caso contrário. Para o exemplo acima, a matriz de adjacência (considerando a ordem alfabética dos vértices) é:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

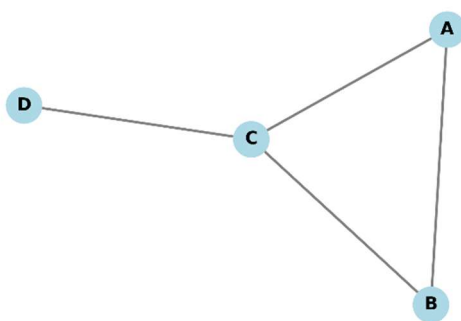


Figura 1 - Grafo Exemplo

2.1.1 Tipos de Grafos

Os grafos podem ser classificados de acordo com a natureza de suas arestas e a estrutura dos vértices. Os principais tipos relevantes para este trabalho são:

- **Grafos não direcionados:** As arestas não têm orientação, ou seja, a relação é simétrica. No

exemplo anterior, a colaboração é mútua, portanto o grafo é não direcionado.

Exemplo - Rede de citações entre artigos

Em uma base de artigos acadêmicos, cada artigo é um vértice. Uma aresta direcionada do artigo A para o artigo B indica que A cita B. Essa relação é assimétrica: A pode citar B sem que B cite A. A partir dessa rede, é possível calcular métricas como o fator de impacto, identificar artigos seminal (muito citados) e estudar a propagação de ideias ao longo do tempo. A matriz de adjacência, nesse caso, não é simétrica.

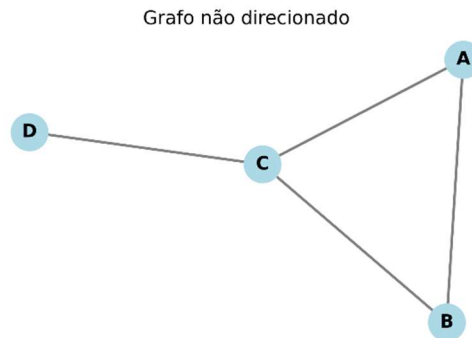


Figura 2 - Grafo não direcionado

- **Grafos direcionados (digrafos):** As arestas possuem uma direção, indicando uma relação assimétrica. Por exemplo, em uma rede de citações, se o artigo A cita o artigo B, existe uma aresta de A para B, mas não necessariamente o contrário. Formalmente, as arestas são pares ordenados (u, v) .

Exemplo - Rede de citações entre artigos

Em uma base de artigos acadêmicos, cada artigo é um vértice. Uma aresta direcionada do artigo A para o artigo B indica que A cita B. Essa relação é assimétrica: A pode citar B sem que B cite A. A partir dessa rede, é possível calcular métricas como o fator de impacto, identificar artigos seminal (muito citados) e estudar a propagação de ideias ao longo do tempo. A matriz de adjacência, nesse caso, não é simétrica.

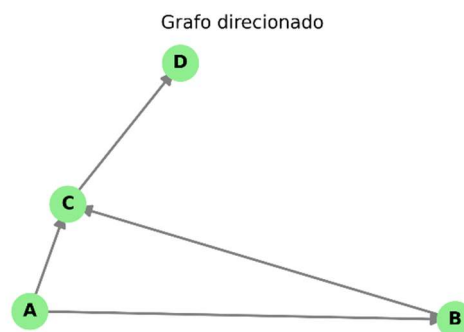


Figura 3 - Grafo direcionado

- **Grafos ponderados:** As arestas possuem um peso associado, que pode representar intensidade, custo, distância, etc. Por exemplo, em uma rede de transportes, o peso pode ser a distância entre cidades. A matriz de adjacência, nesse caso, armazena os pesos: $A_{ij} = w_{ij}$.

Exemplo – Rede de rotas aéreas com fluxo de passageiros

Aeroportos são os vértices, e as arestas representam voos diretos entre eles. Cada aresta possui um peso correspondente ao número médio anual de passageiros transportados nessa rota (ou à distância, ao tempo de voo etc.). Essa rede ponderada permite identificar os aeroportos mais movimentados, otimizar rotas e analisar a resiliência do sistema de transporte aéreo. A matriz de adjacência armazena os pesos em vez de valores binários.

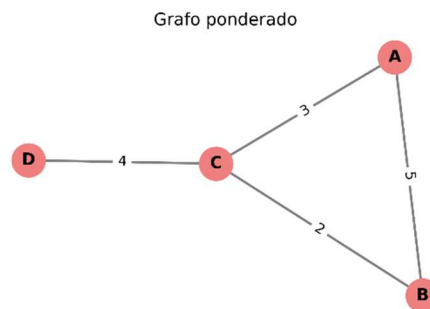


Figura 4 - Grafo ponderado

- **Grafos bipartidos:** Os vértices podem ser particionados em dois conjuntos disjuntos U e V de tal forma que todas as arestas conectam um vértice de U a um vértice de V . Não há arestas entre vértices do mesmo conjunto. Esse tipo de grafo é natural para modelar relações entre dois tipos distintos de entidades, como autores e artigos, ou países e produtos.

Exemplo – Rede de atuação de atores em filmes:

Atores e filmes formam dois conjuntos disjuntos de vértices. Uma aresta conecta um ator a um filme se ele atuou naquele filme. Não há arestas entre atores ou entre filmes diretamente. Essa estrutura permite estudar a colaboração indireta entre atores (atores que atuaram no mesmo filme) e construir projeções como a rede de colaboração entre atores (onde dois atores são ligados se atuaram juntos em pelo menos um filme). É um modelo clássico de grafo bipartido.

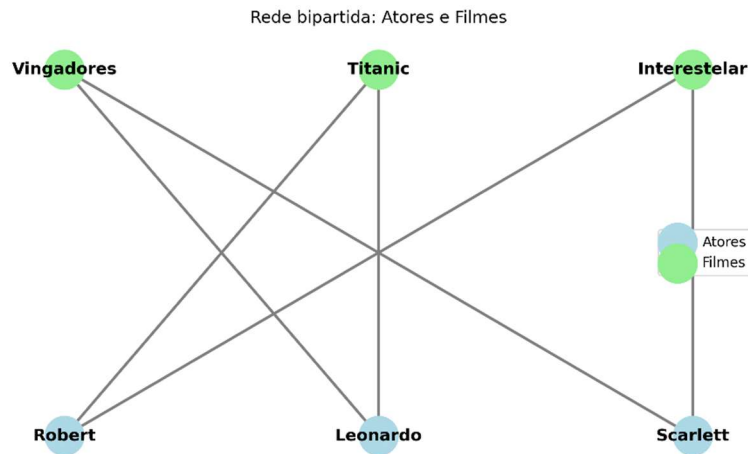


Figura 5 - Grafo bipartido

- **Grafos bipartidos ponderados:** Combinam as duas características anteriores: são grafos bipartidos cujas arestas possuem pesos. Um exemplo é uma rede de compras em que clientes (conjunto U) adquirem produtos (conjunto V) com determinada frequência (peso).

Exemplo – Figura Espaço de Produtos:

O Espaço de Produtos é uma representação econômica que modela a proximidade entre produtos com base na probabilidade de serem exportados juntos por um mesmo país. Pode ser construído a partir de um grafo bipartido ponderado: de um lado, os países; do outro, os produtos. As arestas indicam se um país exporta um produto, e o peso pode ser o volume exportado. A partir dessa estrutura, projeta-se uma rede de produtos onde dois produtos são conectados se são frequentemente coexportados pelos mesmos países. Essa rede resultante é útil para analisar a complexidade econômica e o desenvolvimento de países.

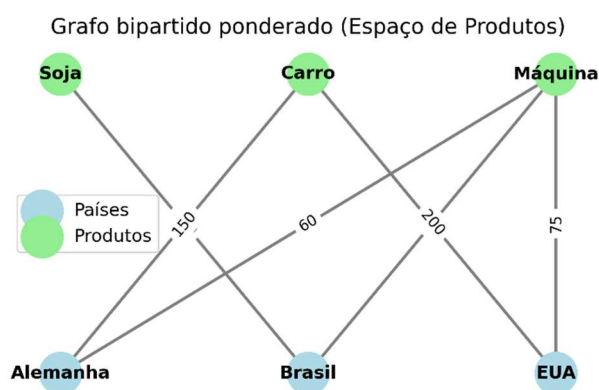


Figura 6 - Grafo espaço de produtos

(Todas as classificações podem ser amparadas com a referência do livro que estou usando. TEORIA COMPUTACIONAL DE GRAFOS – Os Algoritmos Jaime Luiz Szwarcfiter. Referência - Szwarcfiter, J. L. (2018). Teoria computacional de grafos. Elsevier.)

2.1.2 Caminhos e passeios aleatórios

Um caminho em um grafo é uma sequência de vértices $v_1, v_2, \dots, v_k$ tal que cada par consecutivo (v_i, v_{i+1}) é uma aresta. O comprimento do caminho é o número de arestas percorridas. Caminhos são fundamentais para entender a conectividade e a propagação de influência em redes.

Um passeio aleatório (random walk) é um processo estocástico em que um "caminhante" se move ao longo do grafo de forma probabilística e randômica. Em tempo discreto, partindo de um vértice, o caminhante escolhe aleatoriamente uma das arestas incidentes e move-se para o vértice vizinho. Esse processo pode ser descrito por uma matriz de transição P , onde P_{ij} é a probabilidade de ir do vértice j para o vértice i . Para grafos não direcionados e não ponderados, uma escolha comum é $P_{ij} = 1/\text{grau}(j)$ se i e j são vizinhos.

Passeios aleatórios são a base de diversos algoritmos de ranqueamento, como o PageRank, que estima a importância de um vértice pela frequência com que é visitado em um passeio aleatório de longa duração.

2.2 Algoritmos de ranqueamento em grafos

O ranqueamento de vértices em grafos atribui um score a cada nó com base na estrutura de conexões. Diferentes algoritmos exploram diferentes propriedades, como grau, centralidade de autovetor ou passeios aleatórios. A seguir, descrevemos os principais algoritmos utilizados neste trabalho, ilustrando seu funcionamento em uma rede simples.

2.2.1 Centralidade de grau

A centralidade de grau é a métrica mais simples: para um vértice v , o grau de entrada (em grafos direcionados) é o número de arestas que chegam a ele, e o grau de saída é o número de arestas que partem. Em grafos não direcionados, o grau é simplesmente o número de vizinhos. Formalmente, para um grafo com matriz de adjacência A , o grau de entrada de v é $\sum_u A_{uv}$. Essa métrica é usada em algoritmos como o SALSA, que em sua versão básica equivale a uma ponderação dos graus.

2.2.2 PageRank

O PageRank (PR) estima a importância de um vértice como a distribuição estacionária de um passeio aleatório em tempo discreto. A intuição é que um vértice é importante se é apontado por outros vértices importantes. O algoritmo incorpora um fator de amortecimento d (tipicamente 0,85) que representa a

probabilidade de o caminhante continuar seguindo as arestas; com probabilidade $1 - d$, ele salta para um vértice aleatório (distribuição uniforme ou personalizada). A equação fundamental é:

$$\mathbf{r} = d M \mathbf{r} + (1 - d) \mathbf{e}$$

onde $\mathbf{r}$ é o vetor de ranks, M é a matriz de transição estocástica (cada coluna soma 1) e $\mathbf{e}$ é o vetor de personalização (geralmente uniforme). A matriz M é construída a partir da matriz de adjacência: $M_{ij} = 1/\text{grau}(j)$ se há uma aresta de j para i , e 0 caso contrário. Para vértices sem arestas de saída (dead ends), são feitos ajustes para garantir a estocasticidade.

Ilustração: Considere o grafo direcionado simples com vértices A, B, C e arestas: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow A$. A matriz de adjacência é:

$$A = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

Os graus de saída: A:2, B:1, C:1. A matriz de transição M (colunas somam 1) é:

$$M = \begin{bmatrix} 0 & 0 & 1 \\ 0.5 & 0 & 0 \\ 0.5 & 1 & 0 \end{bmatrix}$$

Aplicando o PageRank com $d = 0.85$ e $\mathbf{e}$ uniforme, obtém-se o vetor de ranks após convergência. A figura 2 ilustra esse grafo e os ranks resultantes (os valores seriam calculados iterativamente).

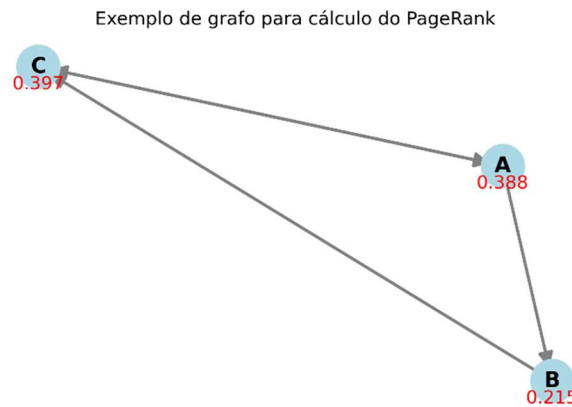


Figura 7 - Grafo exemplo PageRank

2.2.3 HITS (Hyperlink-Induced Topic Search)

O HITS, proposto por Kleinberg (1999), distingue dois papéis para os vértices: **autoridades** (páginas com conteúdo relevante) e **hubs** (páginas que apontam para boas autoridades). A ideia é que uma boa

autoridade é apontada por muitos bons hubs, e um bom hub aponta para muitas boas autoridades. Essa relação de reforço mútuo é expressa por:

$$\mathbf{a} = A^T \mathbf{h}, \mathbf{h} = A \mathbf{a}$$

onde A é a matriz de adjacência (para grafos direcionados). Após normalização, os vetores de autoridade $\mathbf{a}$ e hub $\mathbf{h}$ convergem para os autovetores principais de $A^T A$ e AA^T , respectivamente.

Ilustração: Usando o mesmo grafo anterior, podemos calcular os scores de autoridade e hub. A figura 3 mostra o grafo com os resultados.

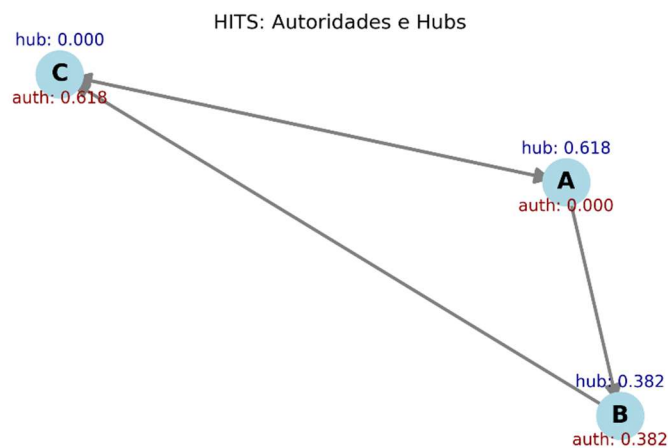


Figura 8 - Grafo hits

2.2.4 SALSA (Stochastic Approach for Link-Structure Analysis)

O SALSA combina ideias do PageRank e do HITS, realizando passeios aleatórios em um grafo bipartido derivado da estrutura de links. Ele alterna entre hubs e autoridades, e sua distribuição estacionária tem uma interpretação simples: em grafos não ponderados, o rank de autoridade de um vértice é proporcional ao seu grau de entrada, e o rank de hub proporcional ao grau de saída. Isso torna o SALSA computacionalmente mais leve que o HITS, além de ser menos suscetível ao efeito de comunidades muito coesas (Tightly Knit Communities).

Ilustração: Para o mesmo grafo, os ranks do SALSA seriam proporcionais aos graus de entrada (autoridade) e saída (hub). A figura 4 ilustra.

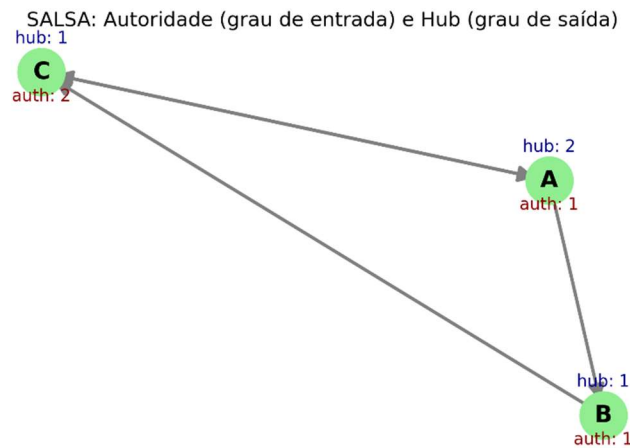


Figura 9 - Grafo SALSA

2.2.5 BiRank e normalizações simétricas

O BiRank é um algoritmo projetado para grafos bipartidos. Diferentemente do PageRank, que usa normalização estocástica, o BiRank adota uma normalização simétrica da matriz de pesos, o que suprime a influência excessiva de vértices de alto grau. A iteração é:

$$\mathbf{r}_{k+1} = \alpha S \mathbf{r}_k + (1 - \alpha) \mathbf{q}$$

onde S é a matriz simetricamente normalizada $S = D_u^{-1/2} W D_v^{-1/2}$, com W a matriz de pesos do grafo bipartido, e D_u, D_v matrizes diagonais de graus. O vetor $\mathbf{q}$ é uma consulta (prior information). O BiRank é flexível para extensões a grafos multipartidos.

2.3 Aplicação das Métricas ao Ranking e Priorização de Requisitos

2.3.1 Fundamentos da priorização de requisitos e desafios

A Engenharia de Requisitos (ER) é um domínio crucial da Engenharia de Software, responsável pela definição e manutenção das necessidades do sistema (Sommerville, 2007). Em projetos de software complexos, a priorização de requisitos é uma atividade essencial para o planejamento eficaz (Karlsson, 2002).

A atividade de rastreabilidade de requisitos estabelece as relações de dependência entre os requisitos,

sendo essa a base estrutural para a priorização (Gotel & Finkelstein, 1994; Sommerville, 2007). A literatura identifica diversas técnicas tradicionais, como a Comparação em Pares (Pair-wise Comparison), que exige que os stakeholders comparem cada par de requisitos, sendo uma tarefa demorada e de alto esforço (Karlsson et al., 2007). Outra técnica é a Votação Acumulativa (100-Points Method), que distribui pontos fixos entre stakeholders, sendo a prioridade final a soma dos pontos recebidos (Ahl, 2005; Leffingwell & Widrig, 2003). Técnicas como o PERT/CPM utilizam grafos para representar a sequência lógica de planejamento, permitindo uma ordenação topológica (Kerzner, 2009). Entretanto, métodos tradicionais de Priorização de Requisitos de Software (PRS) sofrem com problemas intrínsecos de subjetividade e esforço (Firesmith, 2004). Fatores como a inexperiência dos stakeholders, a divergência na interpretação das escalas de prioridade, e o foco excessivo em apenas um ponto de vista resultam em prioridades inconsistentes e "posições inválidas" (Silva et al., 2018). Estudos práticos demonstram que a Votação Acumulativa, por depender excessivamente da ação e da decisão do stakeholder, pode gerar um alto número de posições inválidas e não resolve a duplicidade envolvida em requisitos interdependentes (Silva et al., 2018).

2.3.2 Construção do grafo de requisitos

A aplicação de algoritmos de ranqueamento estrutural na PRS baseia-se na construção de um grafo de requisitos (Silva et al., 2018). Neste modelo, cada requisito é um vértice e as relações de dependência (obtidas pela rastreabilidade) são arestas dirigidas (Silva et al., 2018). Especificamente, se o Requisito A depende de B, um link de avanço se origina em A e aponta para B (Silva et al., 2018).

2.3.3 Mapeamento dos algoritmos de ranqueamento para priorização

Este referencial teórico visa fundamentar a aplicação de algoritmos de ranqueamento de links—notavelmente PageRank, HITS e SALSA—para objetivar e reduzir o esforço na PRS (Silva et al., 2018). Esta abordagem transforma a rede de dependências de requisitos em um grafo estrutural, permitindo que a priorização seja determinada pela topologia da rede, e não predominantemente pela percepção subjetiva do stakeholder (Silva et al., 2018).

O PageRank, em particular, tem se mostrado sólido para a priorização (Silva et al., 2018). Foi usado para medir a complexidade de relacionamentos em sistemas (Li & Yi, 2009) e para analisar o impacto de preocupações em requisitos (Jin et al., 2009).

2.3.4 Benefícios, limitações e considerações práticas

A priorização via PageRank tem se mostrado vantajosa em comparação com métodos manuais como a Votação Acumulativa e PERT/CPM, oferecendo os seguintes benefícios (Silva et al., 2018; He et al., 2014):

1. **Redução da Subjetividade e Esforço:** O PageRank diminui o envolvimento direto dos stakeholders, focando na estrutura de dependência, e é escalável para lidar com um alto número de requisitos, o que é inviável para técnicas manuais (Silva et al., 2018; He et al., 2014).
2. **Solução de Interdependência:** O PageRank, por meio de suas iterações, consegue solucionar a duplicidade envolvida em requisitos interdependentes (mutuamente dependentes), uma limitação não resolvida pela Votação Acumulativa ou PERT/CPM (Silva et al., 2018).
3. **Ajuste Flexível de Prioridade:** É possível introduzir informações externas (como prioridade técnica ou de negócio) no grafo por meio de vértices artificiais (Silva et al., 2018). Essa técnica opcional permite que o rank de um requisito e seu fecho transitivo de dependências sejam ajustados, garantindo que fatores subjetivos necessários sejam incorporados de forma estruturalmente coerente (Silva et al., 2018).

Como consideração prática, embora o PageRank ainda possa resultar em posições inválidas, essa abordagem estrutural se mostrou mais favorável do que as técnicas manuais, apresentando menos inconsistências e maior consistência com a lógica de dependência do sistema (Silva et al., 2018).

As limitações da abordagem incluem as vulnerabilidades topológicas, como o Efeito TKC, que o SALSA foi especificamente desenvolvido para mitigar (Lempel & Moran, 2001).

.

Capítulo 3

Metodologia

O Capítulo 2 apresentam-se recomendações para escrita de uma tese. Na Secção 2.1, são apresentados conceitos basilares. Na Secção 2.2, são identificados os aspectos críticos. Na Secção 2.3, são apresentados avanços na área em estudo.

Conceitos chave: visão geral; conceitos basilares; desafios; avanços; estado da arte.

3.1 Estratégia de Investigação

A investigação adota uma abordagem **quantitativa e experimental** assente em três etapas. Em primeiro lugar, fundamentou-se teoricamente a aplicação de algoritmos de ranqueamento de grafos à Priorização de Requisitos de Software (PRS), conforme exposto no Capítulo 2. Em segundo lugar, projetou-se e implementou-se um protótipo — o **ReqGraph** — capaz de extrair de forma automática o grafo de dependências entre requisitos a partir do código-fonte de uma aplicação Python, recorrendo a análise estática via *Abstract Syntax Tree* (AST). Em terceiro lugar, definiu-se um protocolo experimental para aplicar os algoritmos PageRank, HITS e SALSA aos grafos extraídos de três projetos de código aberto de complexidade crescente, comparando os rankings obtidos.

A opção por análise estática (em detrimento de instrumentação dinâmica) deve-se à reprodutibilidade dos resultados: o grafo extraído depende exclusivamente do código-fonte, sem variabilidade introduzida por entradas de execução. Esta opção é coerente com a literatura sobre rastreabilidade de requisitos, que defende a derivação determinística do grafo de dependências a partir de artefactos estáveis do sistema (Silva et al., 2018).

3.2 Arquitetura do Protótipo ReqGraph

O ReqGraph é o componente que materializa a transformação do código-fonte num **Grafo de Requisitos** $G_R = (V_R, E_R)$, em que cada vértice $v \in V_R$ representa um requisito de domínio e cada aresta direcionada $(r_i, r_j) \in E_R$ traduz uma dependência de implementação detetada entre os requisitos r_i e r_j . O pipeline analítico é composto por quatro estágios sequenciais:

1. **Análise estática via AST** — cada ficheiro Python (.py) do projeto é parseado pelo módulo `ast` da biblioteca padrão. O resultado é uma árvore sintática a partir da qual são extraídas definições de funções (`FunctionDef`), métodos de classes e invocações (`Call`).
2. **Construção do Call Graph** — as invocações detetadas dão origem a um grafo direcionado $G_C = (V_C, E_C)$ em que cada vértice corresponde a uma função (ou método) e cada aresta (f_i, f_j) indica que f_i invoca f_j . O algoritmo resolve referências entre módulos (via `import`), referências a métodos da própria classe (`self.method()`) e qualifica funções pelo seu *namespace* para evitar colisões de nomes.
3. **Mapeamento `func_to_req`** — é fornecido pelo investigador um dicionário que associa cada

função a um requisito de domínio (e.g. `parse_args` → `REQ_PARSING`). Este artefacto é o ponto de contacto entre o nível técnico (funções) e o nível semântico (requisitos). Para reduzir a fricção da sua produção, foi também desenvolvido um *prompt* estruturado (`reqgraph/llm_prompt_mapping.md`) que permite gerar o mapeamento de forma assistida via modelos de linguagem de grande escala (ChatGPT, Claude, Gemini).

4. **Derivação do Grafo de Requisitos** — para cada aresta $(f_i, f_j) \in E_C$ tal que f_i está mapeada para o requisito r_a e f_j está mapeada para o requisito r_b , com $r_a \neq r_b$, é introduzida a aresta (r_a, r_b) em E_R . Arestas duplicadas e auto-laços (correspondentes a invocações internas dentro do mesmo requisito) são descartados.

A Figura 10 ilustra esquematicamente o pipeline.

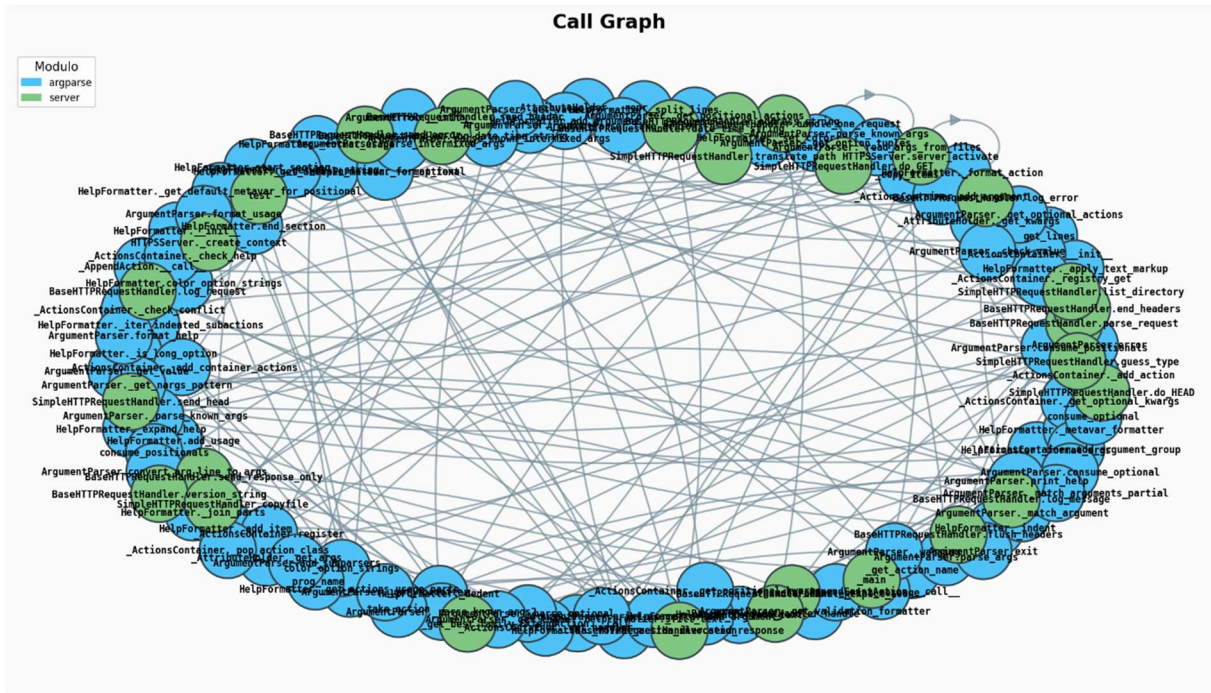


Figura 10 - Exemplo de Call Graph extraído via AST (CPython stdlib)

3.3 Implementação e Tecnologias

O protótipo foi implementado em **Python 3.8+**, organizado como pacote instalável via `pip install -e reqgraph/`. As escolhas tecnológicas seguem critérios de robustez e adesão a *standards* da comunidade científica:

Tecnologia	Versão mínima	Função no pipeline
ast (biblioteca padrão)	—	Parsing sintático do código-fonte Python
networkx	≥ 3.0	Manipulação e análise de grafos direcionados
matplotlib	≥ 3.5	Geração de visualizações em PNG
numpy / scipy	≥ 1.24 / ≥ 1.10	Computação matricial (matrizes de transição do SALSA)
graphviz (DOT)	—	Formato de exportação canônico para grafos

Tabela 1 – Tabela de implementações

A interface de linha de comando (`python -m reqgraph <projeto> --mapping <mapeamento.py>`) expõe o pipeline completo, gerando como artefactos: `call_graph.png`, `req_graph.png`, `req_graph.json` (lista de adjacência) e `req_graph.dot` (formato Graphviz).

3.3.1 Módulo de Raqueamento (ranker.py)

O módulo `ranker.py` consome o ficheiro `req_graph.json` produzido pelo `ReqGraph` e aplica três algoritmos de ranqueamento sobre o grafo direcionado G_R . A semântica adotada para as arestas é: $r_a \rightarrow r_b$ **significa que o requisito r_a depende de r_b** . Em consequência, o *in-degree* de um vértice mede quantos requisitos dele dependem (uma autoridade no sentido de Kleinberg), enquanto o *out-degree* mede quantos outros requisitos ele invoca (um *hub*).

3.3.2 PageRank

A implementação recorre a `nx.pagerank` com parâmetros canónicos: fator de amortecimento $d = 0,85$, número máximo de iterações 100 e tolerância de convergência 10^{-6} . *Dangling nodes* (vértices sem arestas de saída) são tratados internamente pelo `NetworkX` por redistribuição uniforme. A soma dos *scores* totaliza 1,0, permitindo interpretá-los como uma distribuição estacionária de probabilidade.

3.3.3 HITS

A implementação recorre a `nx.hits` com tolerância 10^{-8} e 100 iterações máximas. Retorna dois vetores

normalizados (norma euclidiana unitária): *hubs* e *authorities*. A interpretação para a PRS é direta: um requisito com elevado *authority* é uma dependência crítica do sistema (alterações nele propagam-se a muitos consumidores), enquanto um requisito com elevado *hub* é um orquestrador (alterações afetam muitas integrações).

3.3.4 SALSA

O NetworkX **não fornece** uma implementação do SALSA, pelo que esta foi desenvolvida manualmente seguindo Lempel e Moran (2001). Para cada aresta $(u, v) \in E_R$ constrói-se duas matrizes de transição:

- $H \rightarrow A$: probabilidade $\frac{1}{\text{out}(u)}$ de um *hub* u transitar para a *authority* v .
- $A \rightarrow H$: probabilidade $\frac{1}{\text{in}(v)}$ de uma *authority* v regressar a um *hub* u .

A cadeia de *hubs* tem matriz $T_{hu} = (A \rightarrow H)(H \rightarrow A)$ e a de *authorities* $T_{auth} = (H \rightarrow A)(A \rightarrow H)$. Os *scores* finais são as distribuições estacionárias obtidas por *power iteration* com tolerância 10^{-8} . Linhas nulas (correspondentes a vértices absorventes) são tratadas por redistribuição uniforme, em analogia ao tratamento de *dangling nodes* do PageRank.

A diferença substantiva face ao HITS é a **normalização local** (por grau de cada vértice), que torna o SALSA menos sensível ao **Efeito TKC** (*Tightly Knit Community*) — uma das vulnerabilidades topológicas identificadas no Capítulo 2.

3.3.5 Artefatos Gerados

Para cada projeto analisado, o `ranker.py` produz dois artefactos: `ranking_results.json` (com os *scores* completos dos três algoritmos) e `ranking_results.png` (com gráficos de barras horizontais agrupados por algoritmo). Quando executado com a flag `--all`, gera adicionalmente `ranking_consolidado.json` e `ranking_comparativo.png` na raiz do projeto, agregando os resultados dos três casos de estudo.

3.4 Corpus Experimental e Protocolo de Avaliação

3.4.1 Seleção dos Casos de Estudo

Para validar a abordagem em diferentes regimes de complexidade arquitetural, foram selecionados **três projetos reais de código aberto** disponíveis no GitHub, organizados por nível crescente de acoplamento esperado:

Nível	Projeto	Ficheiros Analisados	Origem
Simples	CPython stdlib	Lib/argparse.py, Lib/http/server.py	python/cpython
Médio	Flask	src/flask/app.py, src/flask/cli.py, src/flask/blueprint s.py	pallets/flask
Complexo	scikit- learn	sklearn/pipeline.py, sklearn/linear_model/_base.py, sklearn/linear_model/_logistic.py, sklearn/tree/_clas ses.py	scikit-learn/scikit- learn

Tabela 2 – Projetos de teste

A seleção privilegia ficheiros representativos do *core* arquitetural de cada projeto, evitando módulos auxiliares (testes, utilitários, *type stubs*). Para cada projeto foi construído um mapeamento `func_to_req` com 9 a 13 requisitos de domínio, identificados a partir da documentação oficial e da inspeção do código-fonte.

3.4.2 Protocolo Experimental

Cada caso de estudo foi processado seguindo os mesmos seis passos, automatizados pelo *script* `run_tests.py`:

1. Cópia dos ficheiros-alvo do repositório de origem para a pasta `testes/<nível>/`.
2. Construção manual do dicionário `func_to_req` em `mapeamento.py`.
3. Execução do ReqGraph: `python -m reqgraph testes/<nível>/ --mapping testes/<nível>/mapeamento.py`.
4. Inspeção dos artefactos gerados (`call_graph.png`, `req_graph.png`, `req_graph.json`).
5. Execução do `ranker.py` sobre o `req_graph.json` gerado.
6. Comparação dos rankings produzidos pelos três algoritmos.

3.4.3 Dimensões de Análise

A análise dos resultados (Capítulo 4) é estruturada em duas dimensões:

- **Resultados Técnicos** — métricas estruturais do pipeline: número de funções analisadas, arestas no Call Graph, requisitos identificados, arestas no Grafo de Requisitos e tempos de convergência dos algoritmos.
- **Resultados Analíticos** — interpretação dos rankings: identificação dos requisitos mais críticos, comparação entre os três algoritmos, detecção de convergências e divergências, e discussão à luz da arquitetura conhecida de cada projeto.

Capítulo 4

Avaliação de Resultados

O Capítulo 3 apresenta as principais conclusões da tese. Na Secção 3.1 apresenta-se um sumário da tese. Na Secção 3.2 apresentam-se as principais contribuições e resultados. Na Secção 3.3 apresentam-se algumas considerações sobre trabalho futuro

Conceitos chave: conclusão; sumário; novidade; resultados principais; trabalho futuro.

4.1 Resultados Técnicos

Esta secção sintetiza as métricas estruturais obtidas em cada caso de estudo, evidenciando a escalabilidade do pipeline e o grau de acoplamento detetado em cada projeto.

4.1.1 Síntese Quantitativa

A Tabela 3 resume as principais métricas obtidas pelo pipeline ReqGraph nos três casos de estudo

Resultados Analíticos

Naturalmente, o seu trabalho pode ser continuado explorando vários tópicos relacionados com o tema da tese. Nesta secção deve apresentar os tópicos

Métrica	CPython stdlib	Flask	scikit-learn
Ficheiros analisados	2	3	4
Funções/métodos detetados	298	125	210
Arestas no Call Graph	130	47	80
Domínios de requisito mapeados	10	13	9
Arestas no Grafo de Requisitos	9	11	17
Densidade do Grafo de Requisitos	0,10	0,13	0,30
Resultado da execução	Passou	Passou	Passou

Tabela 3 – Síntese quantitativa dos três casos de estudo

Observa-se que o número de funções analisadas não cresce monotonamente com a complexidade percebida (o `argparse` da `stdlib` tem mais funções do que o `app.py` do Flask), mas o número de arestas no Grafo de Requisitos sim — passando de 9 (CPython) para 11 (Flask) e 17 (scikit-learn). Esta métrica é, portanto, mais informativa sobre o acoplamento arquitetural do que a contagem bruta de funções.

4.1.2 Caso Simples – Cpython stdlib

Foram analisados os módulos `argparse.py` (parsing de argumentos) e `http/server.py` (servidor HTTP simples). O mapeamento associa 298 funções a 10 requisitos de domínio. O Grafo de Requisitos resultante apresenta apenas 9 arestas, sem ciclos, e pode ser sintetizado pelas seguintes adjacências:

```
{  
  "REQ_ACTIONS":["REQ_CONFIGURATION", "REQ_FORMATTING", "REQ_VALIDATION"],  
  "REQ_ERROR_HANDLING":["REQ_FORMATTING"],  
  "REQ_FORMATTING":["REQ_ERROR_HANDLING"],  
  "REQ_HTTP_HANDLER":["REQ_HTTP_CONTENT", "REQ_HTTP_LOGGING"],  
  "REQ_PARSING":["REQ_ERROR_HANDLING", "REQ_VALIDATION"]  
}
```

O grafo divide-se claramente em dois sub-sistemas independentes (parsing/argparse vs. HTTP), confirmando o baixo acoplamento esperado de módulos da biblioteca padrão. O único ciclo identificado é o par `REQ_FORMATTING ↔ REQ_ERROR_HANDLING`, refletindo a coexistência das funções `format_help` e `error` que se chamam mutuamente.

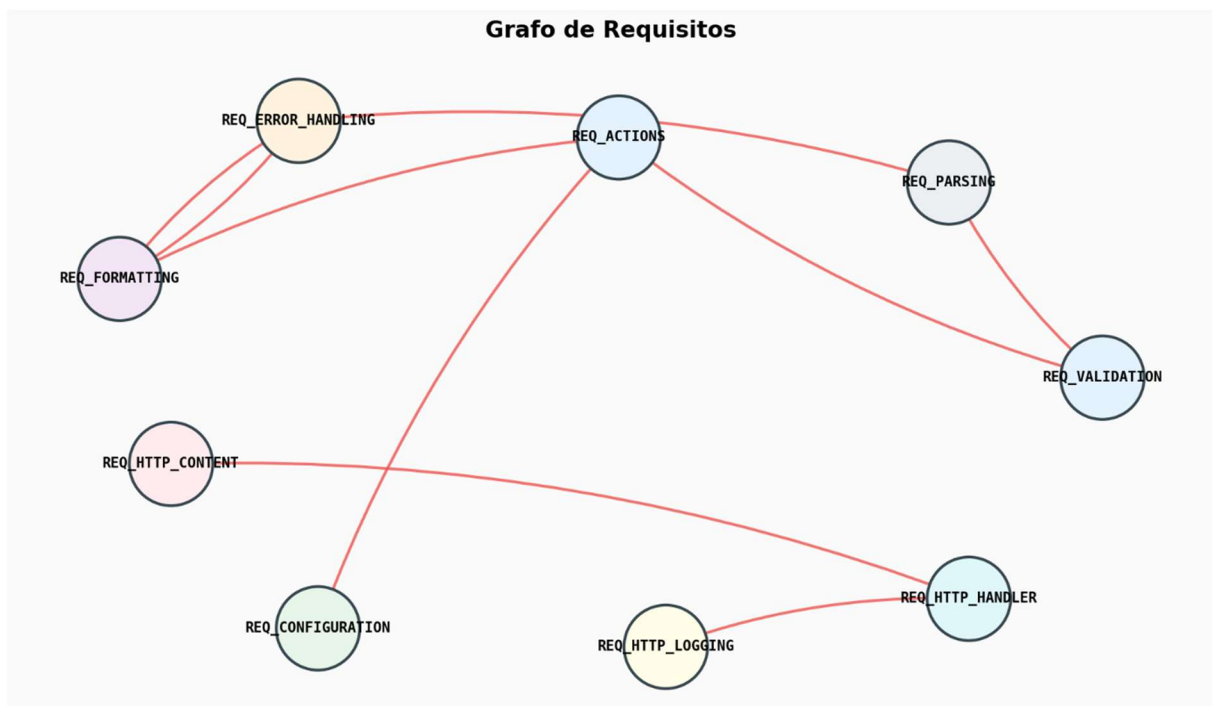


Figura 11 – Grafo de Requisitos: CPython stdlib

4.1.3 Caso Intermediário – Flask

Foram analisados `app.py`, `cli.py` e `blueprints.py`, perfazendo 125 funções mapeadas em 13 requisitos. O Grafo de Requisitos contém 11 arestas e — diferentemente do caso anterior — apresenta **fortes relações cíclicas**:

```
{  
  "REQ_APP_CORE":["REQ_CLI_COMMANDS","REQ_CLI_FRAMEWORK",  
  "REQ_CONTEXT"],  
  "REQ_CLI_FRAMEWORK":["REQ_CONTEXT"],  
  "REQ_CONTEXT":["REQ_REQUEST_HANDLING"],  
  "REQ_ERROR_HANDLING":["REQ_REQUEST_HANDLING"],  
  "REQ_REQUEST_HANDLING":["REQ_CONTEXT","REQ_ERROR_HANDLING",  
  "REQ_ROUTING"],  
  "REQ_ROUTING":["REQ_ERROR_HANDLING", "REQ_REQUEST_HANDLING"]  
}
```

O ciclo `REQUEST_HANDLING ↔ CONTEXT ↔ ROUTING ↔ ERROR_HANDLING` evidencia que estas quatro responsabilidades estão estreitamente acopladas, o que é coerente com a arquitetura *middleware* do Flask, onde o ciclo de vida de uma requisição obriga à coordenação entre dispatcher, contexto de aplicação e tratamento de exceções.

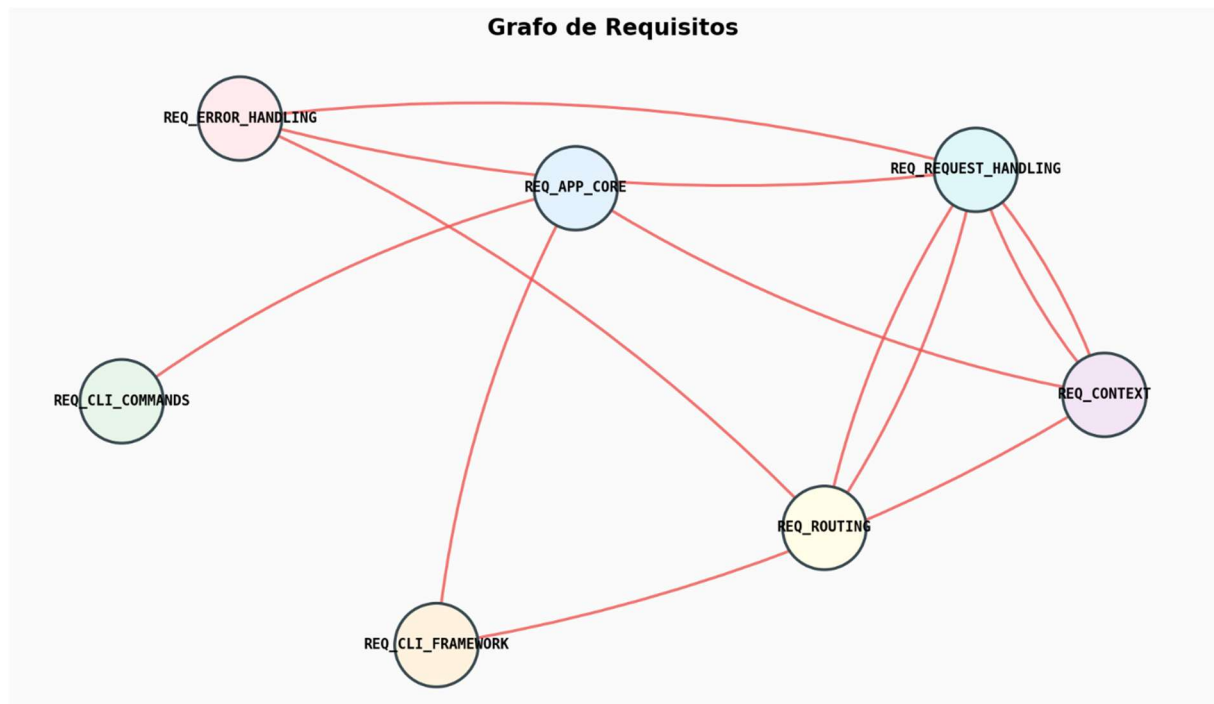


Figura 12 – Grafo de Requisitos: Flask

4.1.4 Caso Complexo – scikit-learn

Foram analisados `pipeline.py`, `linear_model/_base.py`, `linear_model/_logistic.py` e `tree/_classes.py`, contabilizando 210 funções mapeadas em 9 requisitos. O Grafo de Requisitos resultante é o **mais denso** dos três casos (17 arestas), refletindo o elevado grau de orquestração interna do *framework*:

```

{
  "REQ_DECISION_TREE":["REQ_SKLEARN_TAGS"],
  "REQ_FEATURE_UNION":["REQ_PIPELINE"],
  "REQ_LOGISTIC_REGRESSION":["REQ_PIPELINE_PREDICT"],
  "REQ_PIPELINE":["REQ_LOGISTIC_REGRESSION",          "REQ_PIPELINE_PREDICT",
"REQ_SKLEARN_TAGS"],
  "REQ_PIPELINE_FIT":["REQ_DECISION_TREE",          "REQ_FEATURE_UNION",
"REQ_LINEAR_MODEL", "REQ_LOGISTIC_REGRESSION", "REQ_PIPELINE"],
  "REQ_PIPELINE_PREDICT":["REQ_DECISION_TREE",          "REQ_FEATURE_UNION",
"REQ_LINEAR_MODEL", "REQ_LOGISTIC_REGRESSION", "REQ_PIPELINE"],
  "REQ_SKLEARN_TAGS":["REQ_PIPELINE"]
}

```

REQ_PIPELINE_FIT e REQ_PIPELINE_PREDICT conectam-se cada um a cinco outros requisitos, confirmando o seu papel de **orquestradores centrais**. REQ_PIPELINE recebe arestas de quatro requisitos distintos, sendo a **autoridade dominante** do sistema.

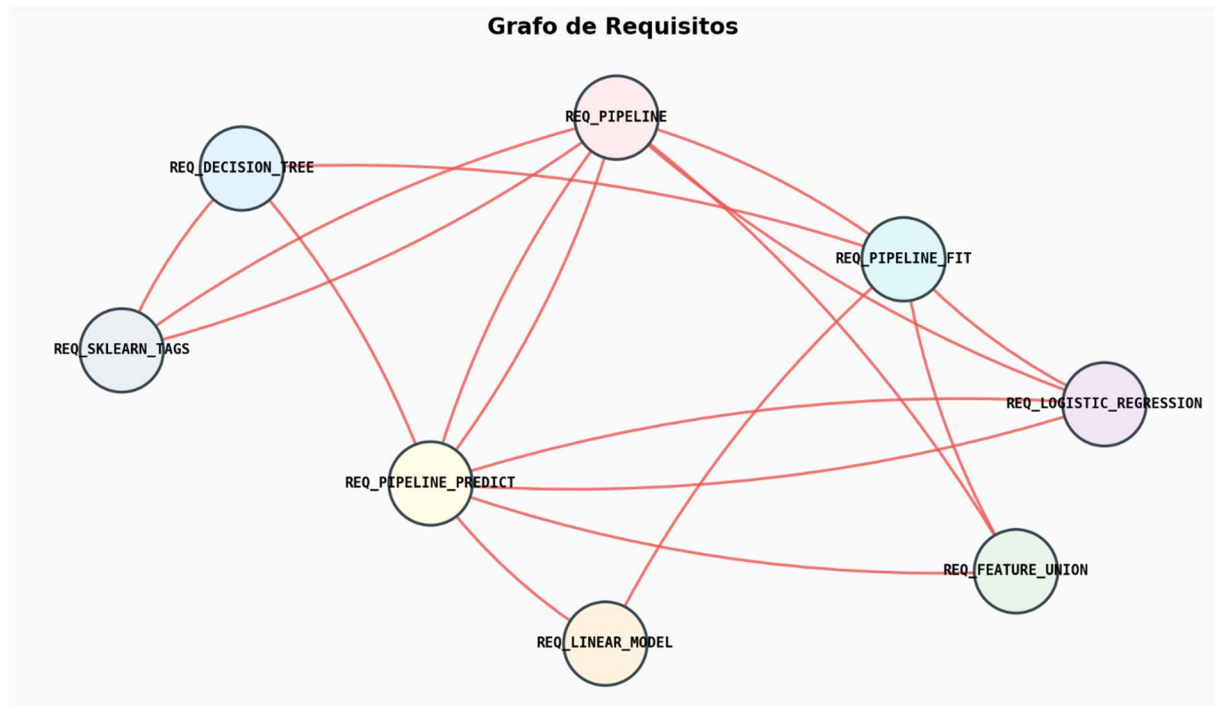


Figura 13 – Grafo de Requisitos: scikit-learn

4.2 Resultados Analíticos

Esta secção apresenta os rankings produzidos pelo PageRank, HITS e SALSA para cada caso de estudo, interpretando os *scores* à luz da arquitetura conhecida dos projetos.

4.2.1 Caso Simples – Rankings para o CPython stdlib

A Tabela 2 reúne os três primeiros requisitos identificados por cada algoritmo no caso simples.

Algoritmo	1°	2°	3°
PageRank	REQ_ERROR_HANDLING (0,337)	REQ_FORMATTING (0,334)	REQ_VALIDATION (0,064)
HITS Authority	REQ_VALIDATION (0,338)	REQ_FORMATTING (0,280)	REQ_CONFIGURATION (0,209)
HITS Hub	REQ_ACTIONS (0,462)	REQ_PARSING (0,285)	REQ_ERROR_HANDLING (0,156)
SALSA Authority	REQ_HTTP_CONTENT (0,167)	REQ_HTTP_LOGGING (0,167)	REQ_ERROR_HANDLING (0,167)
SALSA Hub	REQ_HTTP_HANDLE (0,2)	REQ_ERROR_HANDLING (0,2)	REQ_ACTIONS (0,2)

Tabela 4 – Top-3 por algoritmo: CPython stdlib

O PageRank identifica REQ_ERROR_HANDLING como o requisito mais crítico (concentrando $\approx 33\%$ da massa de probabilidade), seguido de muito perto por REQ_FORMATTING. Esta dupla decorre do ciclo $\text{FORMATTING} \leftrightarrow \text{ERROR_HANDLING}$, que faz a probabilidade acumular-se nesses dois vértices via *random walk*. O HITS distingue claramente os papéis: REQ_ACTIONS e REQ_PARSING são *hubs* (consomem muitas dependências), enquanto REQ_VALIDATION, REQ_FORMATTING e REQ_CONFIGURATION são *authorities* (são consumidos por muitos). O SALSA, devido à sua normalização local, atribui *scores* idênticos a todos os vértices não-absorventes — comportamento esperado num grafo com graus pouco diferenciados.



Figura 14 - Rankings: CPython stdlib

4.2.2 Caso Intermediário – Rankings para o Flask

A Tabela 5 apresenta os top-3 do caso médio.

Algoritmo	1°	2°	3°
PageRank	REQ_REQUEST_HANDLING (0,400)	REQ_ERROR_HANDLING (0,198)	REQ_CONTEXT (0,174)
HITS — Authority	REQ_CONTEXT (0,318)	REQ_ERROR_HANDLING (0,204)	REQ_ROUTING (0,137)
HITS — Hub	REQ_REQUEST_HANDLING (0,319)	REQ_APP_CORE (0,264)	REQ_CLI_FRAMEWORK (0,154)
SALSA — Authority	uniforme a 0,167 entre 6 requisitos	—	—
SALSA — Hub	uniforme a 0,167 entre 6 requisitos	—	—

Tabela 5 - Top-3 por algoritmo: Flask

REQ_REQUEST_HANDLING emerge como o vértice central segundo o PageRank, com 40% da massa total — um valor anormalmente elevado que reflete a sua posição em três arestas de entrada e três de saída, todas dentro do ciclo principal. O HITS divide o protagonismo: REQ_REQUEST_HANDLING é simultaneamente *hub* e *authority* (acoplamento bidirecional típico do *middleware*), enquanto REQ_CONTEXT lidera as *authorities* puras. O SALSA, ao normalizar localmente, dissolve esta concentração e distribui de forma uniforme — evidenciando o **Efeito TKC** mitigado: o ciclo denso entre as quatro responsabilidades seria fortemente premiado pelo HITS, mas o SALSA não cede a essa dominância.



Figura 15 – Rankings: Flask

4.2.3 Caso Complexo – Rankings para scikit-learn

A Tabela 6 apresenta o top-3 do caso mais complexo.

Algoritmo	1°	2°	3°
PageRank	REQ_PIPELINE (0,257)	REQ_PIPELINE_PREDICT (0,218)	REQ_SKLEARN_TAGS (0,156)
HITS — Authority	REQ_PIPELINE (0,217)	REQ_LOGISTIC_REGRESSION (0,200)	REQ_DECISION_TREE (0,177)
HITS — Hub	REQ_PIPELINE_PREDICT (0,360)	REQ_PIPELINE_FIT (0,360)	REQ_PIPELINE (0,096)
SALSA — Authority	uniforme a 0,143 entre 7 requisitos	—	—
SALSA — Hub	uniforme a 0,143 entre 7 requisitos	—	—

Tabela 6 - Top-3 por algoritmo: scikit-learn

Os três algoritmos convergem na identificação de REQ_PIPELINE como o requisito mais central: PageRank atribui-lhe o *score* mais elevado (0,257) e HITS classifica-o como a *authority* dominante. O HITS identifica adicionalmente REQ_PIPELINE_PREDICT e REQ_PIPELINE_FIT como co-líderes dos *hubs* com *scores* idênticos (0,360) — refletindo a sua simetria estrutural (ambos invocam exatamente os mesmos cinco requisitos). O SALSA volta a produzir uma distribuição uniforme entre os vértices com arestas, indicando que, neste corpus, não há vértices destacados pelas suas *frações* de grau.

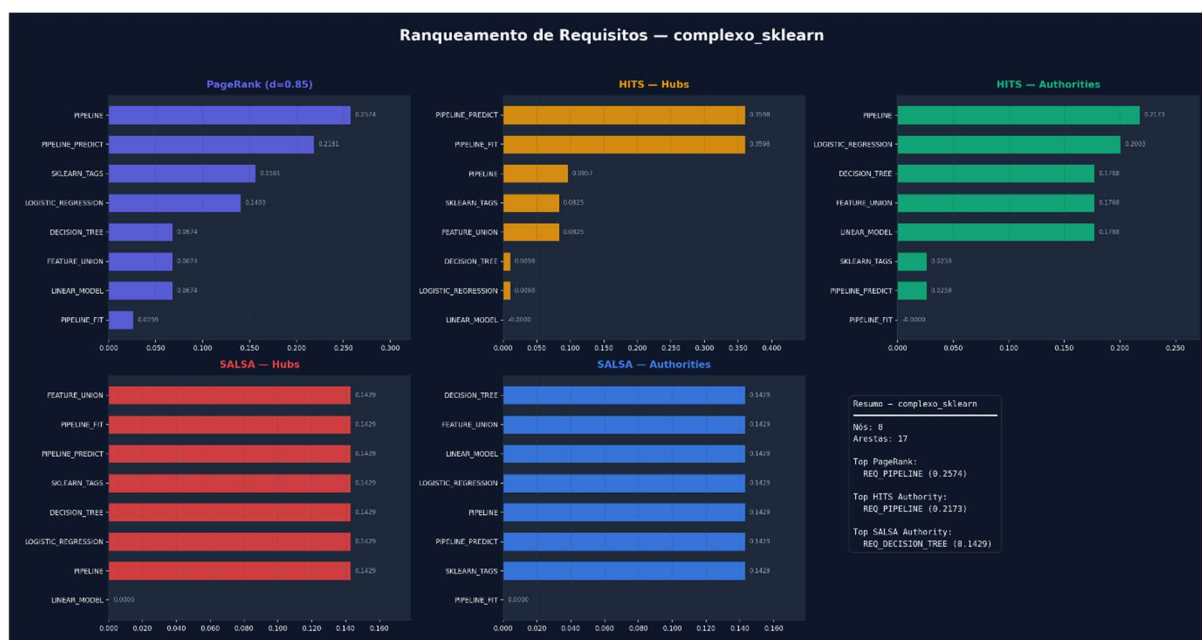


Figura 16 - Rankings: scikit-learn

4.3 Comparação Consolidada

A Figura 17 reúne, num único gráfico, os requisitos *top-1* identificados por cada algoritmo em cada um dos três casos de estudo.

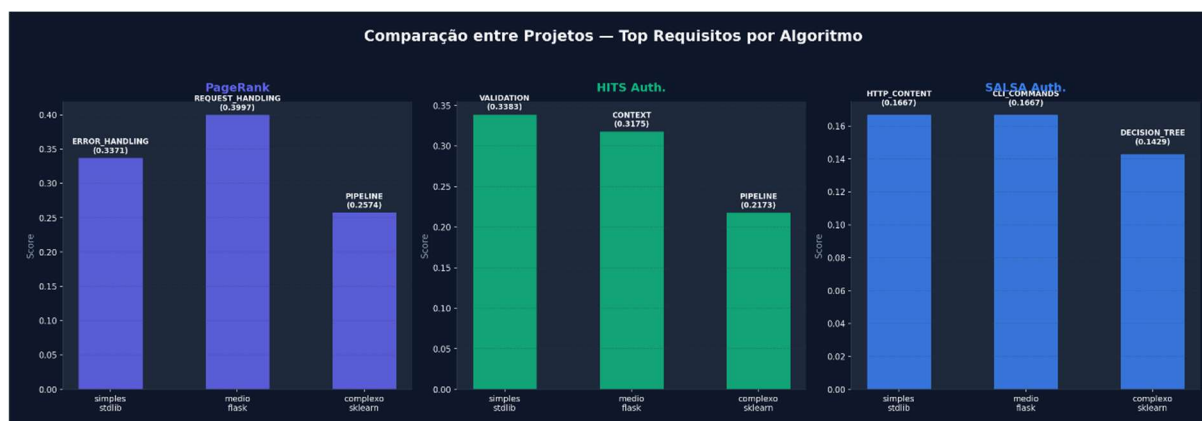


Figura 17 - Ranking comparativo entre os três projetos

4.4 Discussão

A análise cruzada dos resultados permite extrair quatro observações principais:

1. **Convergência no caso complexo (scikit-learn).** Os três algoritmos identificaram REQ_PIPELINE como o requisito mais central. Esta convergência funciona como validação cruzada do método: quando o grafo apresenta uma estrutura suficientemente rica (densidade 0,30), todos os algoritmos detetam o mesmo ponto crítico. Para os engenheiros de software, esta unanimidade aumenta a confiança na prioridade atribuída.
2. **Divergência informativa.** No caso do CPython, o PageRank prioriza REQ_ERROR_HANDLING enquanto o HITS prioriza REQ_VALIDATION (como *authority*) e REQ_ACTIONS (como *hub*). A divergência não é um defeito — é informação suplementar: indica que REQ_ERROR_HANDLING é importante via cadeias transitivas (PageRank), enquanto REQ_VALIDATION é diretamente referenciado por muitos requisitos (HITS *authority*).
3. **Comportamento do SALSA em grafos pequenos.** O SALSA produziu distribuições praticamente uniformes nos três casos. Este resultado é consistente com o teorema de Lempel e Moran (2001), segundo o qual, em grafos fortemente conexos sem pesos, o *score* de *authority* converge para $\ln(v)/|E|$. Em grafos pequenos e relativamente regulares, esta propriedade traduz-se em rankings achatados. A vantagem do SALSA — mitigar o Efeito TKC — só se manifesta em grafos com sub-comunidades densas embutidas em estruturas maiores; o presente corpus, por dimensão, não permite observar este efeito plenamente.
4. **Acoplamento como propriedade emergente.** Os ciclos identificados no Flask e a alta densidade no scikit-learn não foram inseridos manualmente — emergem do código real. Isto sustenta a tese de que o **grafo de dependências contém informação latente** sobre a importância dos requisitos, suficiente para apoiar decisões de priorização sem recorrer exclusivamente à percepção subjetiva dos *stakeholders*.

Em conjunto, estes resultados confirmam a viabilidade da abordagem proposta: o pipeline AST → Call Graph → Req Graph → Ranking é capaz de extrair, de forma totalmente automatizada e reprodutível, um conjunto de prioridades estruturais coerentes com a arquitetura real dos sistemas analisados.

Capítulo 5

Conclusão

O Capítulo 3 apresenta as principais conclusões da tese. Na Secção 3.1 apresenta-se um sumário da tese. Na Secção 3.2 apresentam-se as principais contribuições e resultados. Na Secção 3.3 apresentam-se algumas considerações sobre trabalho futuro

Conceitos chave: conclusão; sumário; novidade; resultados principais; trabalho futuro.

5.1 Sumário e Principais Contribuições

Esta dissertação investigou a viabilidade de aplicar algoritmos de ranqueamento baseados em grafos — concretamente PageRank, HITS e SALSA — à priorização estrutural de requisitos de software. O problema original, exposto no Capítulo 1, partia da constatação de que as técnicas tradicionais de Priorização de Requisitos de Software (PRS), como a Votação Acumulativa e o PERT/CPM, são fortemente dependentes da intervenção subjetiva dos stakeholders e não escalam para sistemas com elevado número de requisitos interdependentes (Silva et al., 2018).

A resposta proposta consistiu em transferir parte do esforço de priorização para a topologia do sistema: se o grafo de dependências contém informação latente sobre a importância relativa dos requisitos, então um algoritmo de ranqueamento adequadamente escolhido pode extrair essa informação de forma reproduzível e automatizada. Para validar esta hipótese, foram desenvolvidos dois artefactos:

- **ReqGraph** — um protótipo Python, organizado como pacote instalável, que automatiza o pipeline **Código-fonte** → **AST** → **Call Graph** → **Grafo de Requisitos**. A análise é totalmente estática (não requer execução do código) e o protótipo gera artefactos em três formatos canónicos (PNG, JSON e DOT/Graphviz).
- **ranker.py** — um módulo de análise que aplica PageRank, HITS e SALSA aos grafos produzidos pelo ReqGraph, gerando *scores* comparáveis, gráficos de barras e relatórios consolidados.

A validação foi realizada com **três projetos reais de código aberto** (CPython stdlib, Flask e scikit-learn), cobrindo um intervalo de complexidade que vai de 9 a 17 arestas no Grafo de Requisitos. Os principais resultados, detalhados no Capítulo 4, sustentam quatro contribuições:

1. **Demonstração da viabilidade da rastreabilidade automática.** O pipeline ReqGraph produziu, sem intervenção humana adicional ao mapeamento `func_to_req`, grafos de requisitos coerentes com a arquitetura conhecida dos três projetos — incluindo a deteção emergente do ciclo `REQUEST_HANDLING` ↔ `CONTEXT` ↔ `ROUTING` ↔ `ERROR_HANDLING` no Flask e da centralidade de `REQ_PIPELINE` no scikit-learn.
2. **Comparação empírica entre PageRank, HITS e SALSA** sobre grafos reais de requisitos. Os algoritmos convergem em casos suficientemente densos (scikit-learn, com `REQ_PIPELINE` no topo dos três rankings) e divergem informativamente em casos esparsos (CPython), permitindo distinguir importância transitiva (PageRank) de importância direta (HITS *authority*) e de orquestração (HITS *hub*).

3. **Implementação manual e documentada do SALSA.** Dado que o NetworkX não fornece SALSA, foi desenvolvida uma implementação completa baseada em matrizes de transição de cadeias de Markov, com tratamento explícito de vértices absorventes. Esta implementação é reaproveitável em contextos académicos e industriais.
4. **Mecanismo de redução do custo de mapeamento.** O ficheiro `llm_prompt_mapping.md` formaliza um *prompt* estruturado para gerar o dicionário `func_to_req` via modelos de linguagem de grande escala, mitigando uma das fricções identificadas — a necessidade de mapear manualmente centenas de funções para domínios de requisito.

Os objetivos específicos enunciados na Secção 1.3.2 podem agora ser revisitados:

Objetivo	Estado
Mapeamento Teórico-Matemático dos algoritmos	Capítulo 2
Avaliação de Eficácia na PRS	Capítulo 4 (3 casos de estudo)
Análise de Vulnerabilidades Topológicas (TKC)	Discutido teoricamente; corpus insuficiente para observar o efeito plenamente
Modelo Híbrido com Vértices Artificiais	Identificado como trabalho futuro
Validação por Emulação	Pipeline ReqGraph + 3 projetos reais

Tabela 7 – Objetivo / Estado

A consistência entre as conclusões do referencial teórico e os resultados experimentais sustenta a tese de que os algoritmos de ranqueamento estrutural constituem um complemento — não um substituto — às técnicas tradicionais de priorização, contribuindo para a redução do viés subjetivo em sistemas de larga escala.

5.2 Trabalho Futuro

A investigação realizada abre várias linhas de continuação. Listam-se de seguida as mais relevantes, ordenadas por proximidade ao trabalho desenvolvido:

- **Modelo Híbrido com Vértices Artificiais.** Implementar a técnica de injeção de prioridades de negócio através de vértices artificiais conectados ao grafo de requisitos (Silva et al., 2018). Esta extensão permitirá medir empiricamente o impacto de prioridades subjetivas sobre os rankings estruturais e oferecer um mecanismo de ajuste fino aos *stakeholders*.
- **Análise empírica do Efeito TKC.** Construir corpora sintéticos ou recolher projetos com sub-comunidades densas conhecidas (por exemplo, *frameworks* de plug-ins) para observar empiricamente a vantagem do SALSA sobre o HITS. A presente dissertação confirmou a propriedade teoricamente, mas não a observou em corpora reais.
- **Extensão a outras linguagens.** O motor de extração de Call Graph baseia-se exclusivamente no módulo *ast* do Python. Estender o ReqGraph a Java (via *JavaParser*), TypeScript (via *TypeScript Compiler API*) ou C/C++ (via *libclang*) permitiria validar a generalidade da abordagem.
- **Métricas de validação humana.** A avaliação atual é estrutural e qualitativa. Um estudo controlado com engenheiros de software, comparando rankings automáticos com rankings produzidos por especialistas, forneceria uma medida quantitativa de adequação (e.g., correlação de Spearman).
- **Integração contínua e *snapshots* temporais.** Aplicar o ReqGraph a *snapshots* sucessivos do mesmo projeto (e.g., *tags* de *releases*) permitiria estudar a **evolução estrutural** dos requisitos ao longo do tempo, detetando *drift* arquitetural.
- **Algoritmos adicionais.** Comparar PageRank/HITS/SALSA com o BiRank (apresentado na Secção 2.2.5) e com técnicas baseadas em *graph neural networks*, explorando o potencial de aprendizagem supervisionada quando rótulos de prioridade estão disponíveis.
- **Tratamento de pesos semânticos.** Atualmente, todas as arestas têm peso unitário. Atribuir pesos derivados da frequência de invocação ou da força do acoplamento (e.g., número de funções partilhadas entre dois requisitos) poderá refinar substancialmente os rankings.

O conjunto destas linhas confirma que, embora o problema central tenha sido endereçado, a abordagem proposta abre um espaço de investigação largo no cruzamento entre Engenharia de Requisitos, Teoria dos Grafos e Análise Estática de Código.

Bibliografia

- Ahl, V. (2005). An experimental comparison of five prioritization methods (Master's thesis). Blekinge Institute of Technology.
- Ding, C., He, X., Husbands, P., Zha, H., & Simon, H. D. (2002). PageRank, HITS and a unified framework for link analysis. In *Proceedings of the 25th ACM SIGIR Conference* (pp. 353–354).
- Durrett, R. (2010). *Probability: Theory and examples* (4th ed.). Cambridge University Press.
- Firesmith, D. (2004). Prioritizing requirements. *Journal of Object Technology*, 3(8), 35–48.
- Gikhman, I. I., & Skorokhod, A. V. (1969). *Introduction to the theory of random processes*. W. B. Saunders Company.
- Gotel, O., & Finkelstein, C. (1994). An analysis of the requirements traceability problem. In *Proceedings of the First International Conference on Requirements Engineering* (pp. 94–101). IEEE.
- He, X., Chen, T., Kan, M.-Y., & Chen, X. (2015). TriRank: Review-aware explainable recommendation by modeling aspects. In *Proceedings of the ACM CIKM Conference* (pp. 1661–1670).
- He, X., Gao, M., Kan, M.-Y., & Wang, D. (2014). BiRank: Towards ranking on bipartite graphs. *IEEE Transactions on Knowledge and Data Engineering*, 26(11), 2673–2687.
- Jin, Y., Li, T., & Liu, S. (2009). Applying PageRank algorithm in requirement concern impact analysis. In *33rd Annual IEEE International Computer Software and Applications Conference* (pp. 361–366). IEEE.
- Karlsson, J. (2002). Software requirements prioritizing. In *Proceedings of the Second International Conference on Requirements Engineering* (pp. 110–116). IEEE.
- Karlsson, L., Berander, P., & Ågren, J. (2007). Pair-wise comparisons versus planning game partitioning: Experiments on requirements prioritisation techniques. *Empirical Software Engineering*, 12(1), 3–33.

Kerzner, H. (2009). Project management: A systems approach to planning, scheduling, and controlling. Wiley.

Kleinberg, J. M. (1999). Authoritative sources in a hyperlinked environment. *Journal of the ACM*, 46(5), 604–632.

Kolmogorov, A. N. (1931). Über die analytischen Methoden in der Wahrscheinlichkeitsrechnung. *Mathematische Annalen*, 104(1), 415–458.

Leffingwell, D., & Widrig, D. (2003). Managing software requirements: A use case approach. Pearson Education.

Lempel, R., & Moran, S. (2001). SALSA: The stochastic approach for link-structure analysis. *ACM Transactions on Information Systems*, 19(2), 131–160.

Li, F., & Yi, T. (2009). Apply PageRank algorithm to measuring relationship's complexity. In *PACIIA 2008 Pacific-Asia Workshop on Computational Intelligence and Industrial Application* (Vol. 1, pp. 914–917). IEEE.

Lourenço, H. R., Martin, O. C., & Stützle, T. (2009). Iterated local search. In M. Gendreau & J.-Y. Potvin (Eds.), *Handbook of metaheuristics* (2nd ed., pp. 129–169). Springer.

Masuda, N., Porter, M. A., & Lambiotte, R. (2017). Random walks and diffusion on networks. *Physics Reports*, 716–717, 1–58.

Newman, M. E. J. (2010). *Networks: An introduction*. Oxford University Press.

Ng, A. Y., Zheng, A. X., & Jordan, M. I. (2001). Stable algorithms for link analysis. In *Proceedings of the ACM SIGIR Conference* (pp. 258–266).

Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank citation ranking: Bringing order to the web (Technical Report). Stanford InfoLab.

Ross, S. M. (2014). *Introduction to probability models* (11th ed.). Academic Press.

Silva, M. P., Tirelo, F., & Marques Neto, H. T. (2018). Uso do algoritmo PageRank para priorização de requisitos de software. In *Anais do Congresso Brasileiro de Software: Teoria e Prática*.

Sommerville, I. (2007). *Engenharia de software* (8ª ed.).

Szwarcfiter, J. L. (2018). *Teoria computacional de grafos*. Elsevier.

- Yu, J., Wang, J., & Zhang, J. (2021). A simulated annealing with restart strategy for the path cover problem with time windows. In *International Conference on Advanced Computational Methods in Engineering* (pp. 1–10). Springer.
- Zhou, D., Bousquet, O., Lal, T. N., Weston, J., & Schölkopf, B. (2004). Learning with local and global consistency. In *Advances in Neural Information Processing Systems* (pp. 321–328).



UNIVERSIDADE
LUSÓFONA

www.ulusofona.pt

Glossário

<i>AST (Abstract Syntax Tree)</i>	Representação em árvore da estrutura sintática de um programa, usada como ponto de partida para análise estática de código-fonte.
<i>Call Graph</i>	Grafo direcionado em que cada vértice é uma função (ou método) e cada aresta (f_i, f_j) indica que f_i invoca f_j .
<i>Grafo de Requisitos</i>	Grafo direcionado derivado do Call Graph através de um mapeamento <code>func_to_req</code> , em que cada vértice é um requisito de domínio e as arestas representam dependências entre requisitos.
<i>PageRank</i>	Algoritmo de ranqueamento que modela a probabilidade estacionária de um navegador aleatório visitar cada vértice do grafo.
<i>HITS</i>	Algoritmo de Kleinberg (1999) que atribui a cada vértice dois scores: hub (orquestrador) e authority (dependência central).
<i>SALSA</i>	Algoritmo de Lempel e Moran (2001) que combina HITS com cadeias de Markov sobre um grafo bipartido, normalizando localmente por grau.
<i>Efeito TKC</i>	Tightly Knit Community Effect: vulnerabilidade do HITS em que sub-comunidades densas concentram desproporcionalmente os scores; o SALSA foi projetado para mitigá-lo.
<i>Dangling Node</i>	Vértice sem arestas de saída; no PageRank é tratado por redistribuição uniforme da sua massa de probabilidade.

Apêndice A

Apêndice ou Anexo?

O Apêndice A explica a diferença entre apêndice e anexo.

A.1 Apêndice

Apêndices englobam materiais elaborados pelo autor(a) tais como gráficos, quadros, tabelas, traduções, organogramas e esquemas que prestem informação relevante para a compreensão do trabalho. Só devem figurar nos apêndices as informações previamente referenciadas no texto. As informações são total ou parcialmente da responsabilidade do autor.

A.2 Anexo

Anexos englobam documentos, que não sendo elaborados pelo autor, serviram de base para a construção do estudo, ou facilitam a compreensão da tese/dissertação. Só devem figurar nos anexos documentos e/ou materiais previamente referenciados no corpo do trabalho. Todos os documentos devem estar em formato digital.